Kalm Language Specification

VERSION	0.1
STATUS	DRAFT
DATE	2026-10-01
JAVA VERSION	25

Contents

- 1. What this PoC should prove
- 2. Demo experience
- 3. Language subset
- 4. Compiler outline
- 5. Repository shape
- 6. Build and test gates
- 7. Implementation sequence
  - Milestone 0: Prove the Java/Kalm cycle
  - Milestone 1: Kalm-to-Java vertical slice
  - Milestone 2: Same-module interop
  - Milestone 3: Maven sample
  - Milestone 4: Stabilize and document
- 8. PoC estimate and exit decision
- References

1. What this PoC should prove

Build the smallest end-to-end slice that validates the riskiest promise in the Kalm specification: Kalm and Java sources can call one another inside the same Maven module, using the project's Java dependencies and producing ordinary Java class files.

The PoC is successful when a developer can build the sample project through Maven, compile `.java` and `.kalm` files together, and run the resulting application on JDK 25. The compiler is a Java program that translates Kalm to Java source and uses the standard Java compiler APIs. It does not create a VM or a new bytecode format.

This PoC is a feasibility gate, not a miniature implementation of every 1.x feature. Keep its syntax, type rules, diagnostics, runtime, and editor support deliberately small.

2. Demo experience

The demo contains `src/main/kalm/demo/App.kalm`, `src/main/kalm/demo/Welcome.kalm`, and `src/main/java/demo/JavaGreeter.java` in one Maven project.

`Welcome.kalm`:

```kalm
space demo;
use java.time.LocalDate;

pub class Welcome {
    fun greet(name: Text) -> Text = @"Hello, $name. Today is ${LocalDate.now()}!";
}
```

`App.kalm`:

```kalm
space demo;
use org.apache.commons.lang3.StringUtils;

pub fun greetingForJava(name: Text) -> Text {
    var result: Text = "Welcome";
    result = Welcome().greet(name);
    return result;
}

pub fun main() -> Unit {
    fixed visitor = "Mira";
    if (visitor.isEmpty()) {
        sayLine("Hello there!");
    } else {
        sayLine(Welcome().greet(visitor));
    }
    sayLine(StringUtils.capitalize(JavaGreeter.tagline()));
    sayLine(JavaGreeter.callKalm());
}
```

`JavaGreeter.java`:

```java
package demo;

public final class JavaGreeter {
    private JavaGreeter() {}

    public static String tagline() {
        return "A Java helper, called from Kalm.";
    }

    public static String callKalm() {
        return AppKalm.greetingForJava("Java");
    }
}
```

A source file named `App.kalm` generates the `AppKalm` facade. Its top-level public functions are available to Java through that facade, which also provides the Java `main(String[])` entry point for the Kalm `main` function.

The demo must run through the Maven lifecycle, for example:

```sh
mvn clean verify
mvn exec:java -Dexec.mainClass=demo.AppKalm
```

The readme must list the JDK 25 prerequisite and show the expected output. The POM includes a pinned `org.apache.commons:commons-lang3` dependency; `App.kalm` imports and calls its `StringUtils` type to prove that Kalm uses ordinary Maven-resolved libraries. The Java helper calls `AppKalm.greetingForJava`, and Kalm calls back into that helper so both directions run in the same demo.

3. Language subset

Implement only the syntax and semantics required by the sample and its tests.

Include in the PoC	Defer until after the PoC
UTF-8 `.kalm` files, `space`, `use`, Java-style braces, required semicolons	Gradle plugin and Gradle test integration
Top-level `pub fun`, simple `class`, methods, implicit no-argument construction	Contracts, inheritance, sealed hierarchies, `needs`, `override`
`Text`, `Bool`, `Int`, `Long`, `Unit`, and qualified Java types	Generics, collection literals, full Java generic variance/wildcard model
`fixed` and `var` locals; typed parameters and results	`data`, `choice`, `singleton`, annotations, accessors, delegates
Literals, interpolation, calls, member access, `return`, expression-bodied functions	`async`, `await`, tasks, checked-exception redesign, resource/defer features
A small `if`/`else` statement and basic arithmetic if needed by tests	Null operators, pattern matching, extensions, named/default arguments
Java constructors and public static/instance method calls	Full overload diagnostics, annotation processors, JPMS edge cases
CLI `kalm compile` and useful source locations	`kalm test`, language server, VS Code and IntelliJ plugins

Kalm statements retain Java-style braces and semicolons. The PoC should use the current Kalm vocabulary (`space`, `use`, `class`, `fun`, `fixed`, `var`, `pub`) and avoid introducing temporary keywords that would need migration.

For the PoC, Java's compiler is the final authority for checking generated Java and Java interop calls. The Kalm front end still reports its own syntax and basic name/type errors. Implement `sayLine(Text)` as a compiler builtin that emits `System.out.println(...)`; this avoids a separate runtime artifact. Do not spend the first milestone reproducing all of `javac`'s overload-resolution rules.

4. Compiler outline

Implement the compiler in Java 25 using public JDK APIs only. Use a small hand-written lexer and recursive-descent parser for the subset above; keep syntax nodes separate from code generation so later language changes do not become string substitutions.

A compilation runs these steps:

1. Read Kalm and Java source roots, project classpath, target release, and output directories from the Maven plugin.
2. Lex and parse Kalm. Produce source-spanned syntax nodes and collect Kalm declarations.
3. Generate temporary Java signature stubs for Kalm public declarations needed by Java source analysis.
4. Use `JavacTask.parse()` and `JavacTask.analyze()` on Java sources plus stubs. Collect Java symbols and compiler diagnostics; do not generate or package stub classes.
5. Check Kalm references against its own symbols and the Java language model, then emit real Java sources.
6. Hand off the original Java sources and generated Java sources for final compilation with `--release 25`. In CLI mode, the compiler invokes `javac`; in Maven mode, it registers the generated source root and lets the normal Maven Java compiler compile both source sets. Signature stubs never enter the artifact.

The stub and final-compilation sequence is the first technical risk to test. Keep it isolated behind a small compiler service and prove that a stub is used only for analysis and never appears in the jar. Do not depend on `com.sun.tools.javac.*` internals.

Generated Java is written under `target/generated-sources/kalm/`. The compiler should provide a diagnostic with the original `.kalm` file and source range for lexer/parser errors. For errors reported against generated Java, include the generated file path in the PoC; complete Kalm-to-Java source maps can follow later.

5. Repository shape

Start with a small Maven multi-module repository:

```text
kalm-poc/
  pom.xml
  kalm-compiler/
  kalm-maven-plugin/
  demo-mixed-project/
  docs/
```

- `kalm-compiler`: lexer, parser, syntax tree, basic checking, Java emitter, Java compiler integration, CLI.
- `kalm-maven-plugin`: `kalm:compile` goal, source-root configuration, dependency/classpath and release wiring, generated-source registration.
- `demo-mixed-project`: the Java/Kalm sample and executable acceptance test.
- `docs`: this brief, quick-start, and known limitations.

Do not split the compiler into many artifacts yet. Keep the compiler's command-line entry point callable from the plugin so the CLI and Maven use the same implementation.

6. Build and test gates

All PoC CI runs on JDK 25 and compiles generated Java with `--release 25`.

1. **Lexer/parser tests:** accept every demo construct; reject unterminated strings, missing semicolons, malformed declarations, and unmatched braces with source locations.
2. **Code generation tests:** compile generated Java for each supported Kalm fixture; inspect a few representative generated sources.
3. **Mixed-source test:** in one Maven main source set, Kalm calls `JavaGreeter.tagline()` and Java calls `AppKalm.greetingForJava(...)`.
4. **Java library test:** Kalm imports and invokes a type from a normal Maven dependency.
5. **Build lifecycle test:** `mvn clean verify` works from a clean checkout; the final artifact contains ordinary Java classes and no signature stubs.
6. **Runtime test:** run the sample and assert its output. Also verify a Kalm syntax/type error fails Maven with a non-zero status and a readable diagnostic.

A build only passes if the demo works from a clean checkout. Do not count a manually pre-generated Java file as success.

7. Implementation sequence

Milestone 0: Prove the Java/Kalm cycle

Before implementing the language, create a tiny experiment with one Java class calling a temporary Kalm stub and one Kalm implementation calling that Java class. Confirm the JavacTask analysis phase and final compile can produce the expected class files without packaging stubs. If this sequence fails, revise the joint-compilation design before expanding the compiler.

Milestone 1: Kalm-to-Java vertical slice

Implement the lexer, parser, source spans, basic declarations, expression emitter, and CLI for a Kalm-only hello-world file. Run generated Java through JDK 25 `javac` and execute it.

Milestone 2: Same-module interop

Add Java signatures, Kalm stubs, Java model lookups, and the two-way mixed-source test. Implement the initial supported constructors and method calls.

Milestone 3: Maven sample

Add the Maven goal, classpath/release/source-root wiring, generated-source registration, and the `demo-mixed-project`. Test both `mvn clean verify` and the documented run command.

Milestone 4: Stabilize and document

Add regression fixtures for supported syntax and negative cases, publish the known limitations, and make the quick-start work from a clean clone. Stop here for the first PoC; decide what to add to an alpha after using it on a real mixed project.

8. PoC estimate and exit decision

For one developer comfortable with Java and AI coding tools, budget roughly **2-4 focused weeks** for the whole PoC. The Java/Kalm joint-compilation spike should take only a few days and is the go/no-go gate; parser and Java emission are comparatively routine. The schedule assumes the feature deferrals in Section 3 are held and does not include Gradle, editor plugins, distribution, or 1.x completeness.

At the end, answer these questions from the working demo:

- Can Kalm and Java call each other in the same Maven module?
- Are project Java dependencies visible to Kalm without custom wrappers?
- Can the compiler produce readable errors and ordinary class files from a clean build?
- Is the syntax pleasant enough to justify expanding the implementation?

If the mixed-compilation test is unreliable, stop feature work and fix that foundation first. If the demo is stable, use the sample as the acceptance fixture for the next small milestone rather than implementing the full language at once.

References

- JDK 25 `JavacTask` API
- JDK 25 Java compiler API
- Maven guide to generating sources